networks. This is because of the *scale invariance* in network weights. When ReLU non-linearities are used, the network remains unchanged if we (for example) multiply the weights in one layer of a network by 10, and divide the next layer by 10. This invariance is even more prominent when batch normalization is used. In this case, the size (i.e., norm) of a filter is irrelevant because the output of each layer is re-scaled during batch normalization. For this reason, a network's behavior remains unchanged if we re-scale the weights. Note, this scale invariance applies only to rectified networks.

Scale invariance prevents us from making meaningful comparisons between plots, unless special precautions are taken. A neural network with large weights may appear to have a smooth and slowly varying loss function; perturbing the weights by one unit will have very little effect on network performance if the weights live on a scale much larger than one. However, if the weights are much smaller than one, then that same unit perturbation may have a catastrophic effect, making the loss function appear quite sensitive to weight perturbations. Keep in mind that neural nets are scale invariant; if the small-parameter and large-parameter networks in this example are equivalent (because one is simply a rescaling of the other), then any apparent differences in the loss function are merely an artifact of scale invariance. This scale invariance was exploited by Dinh et al. [7] to build pairs of equivalent networks that have different apparent sharpness.

To remove this scaling effect, we plot loss functions using filter-wise normalized directions. To obtain such directions for a network with parameters θ, we begin by producing a random Gaussian direction vector d with dimensions compatible with θ. Then, we normalize each filter in d to have the same norm of the corresponding filter in θ. In other words, we make the replacement $d_{i,j} \leftarrow \frac{d_{i,j}}{\|d_{i,j}\|}\|\theta_{i,j}\|$, where $d_{i,j}$ represents the jth filter (not the jth weight) of the ith layer of d, and $\|\cdot\|$ denotes the Frobenius norm. Note that the filter-wise normalization is different from that of [21], which normalize the direction without considering the norm of individual filters. Note that filter normalization is not limited to convolutional (Conv) layers but also applies to fully connected (FC) layers. The FC layer is equivalent to a Conv layer with a 1×1 output feature map and the filter corresponds to the weights that generate one neuron.

Do contour plots of the form (1) capture the natural distance scale of loss surfaces when the directions δ and η are filter normalized? We answer this question to the affirmative in Section 5 by showing that the sharpness of filter-normalized plots correlates well with generalization error, while plots without filter normalization can be very misleading. In Appendix A.2, we also compare filter-wise normalization to layer-wise normalization (and no normalization), and show that filter normalization produces superior correlation between sharpness and generalization error.

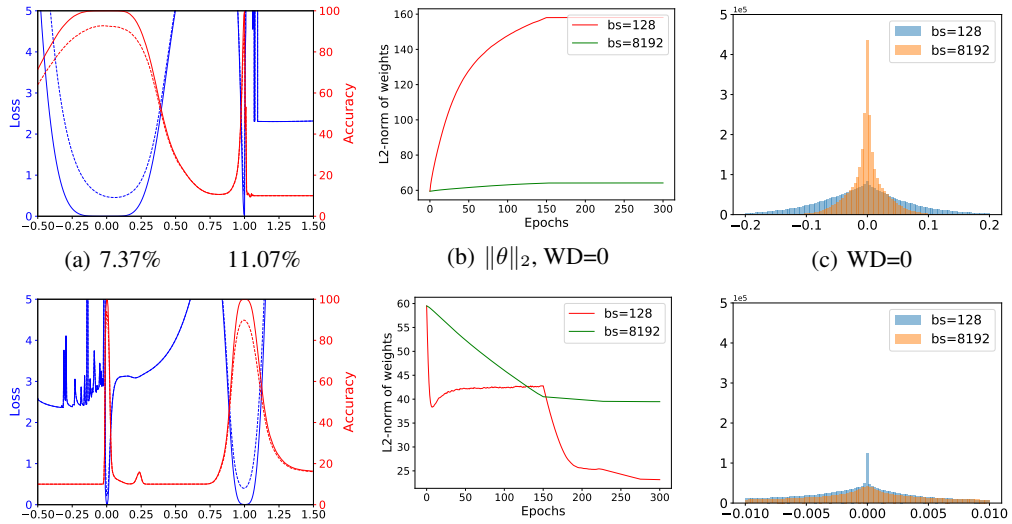
5 The Sharp vs Flat Dilemma

Section 4 introduces the concept of filter normalization, and provides an intuitive justification for its use. In this section, we address the issue of whether sharp minimizers generalize better than flat minimizers. In doing so, we will see that the sharpness of minimizers correlates well with generalization error when filter normalization is used. This enables side-by-side comparisons between plots. In contrast, the sharpness of non-normalized plots may appear distorted and unpredictable.

It is widely thought that small-batch SGD produces "flat" minimizers that generalize well, while large batches produce "sharp" minima with poor generalization [3, 24, 18]. This claim is disputed though, with Dinh et al. [7], Kawaguchi et al. [23] arguing that generalization is not directly related to the curvature of loss surfaces, and some authors proposing specialized training methods that achieve good performance with large batch sizes [19, 14, 6]. Here, we explore the difference between sharp and flat minimizers. We begin by discussing difficulties that arise when performing such a visualization, and how proper normalization can prevent such plots from producing distorted results.

We train a CIFAR-10 classifier using a 9-layer VGG network [33] with batch normalization for a fixed number of epochs. We use two batch sizes: a large batch size of 8192 (16.4% of the training data of CIFAR-10), and a small batch size of 128. Let θ^s and θ^l indicate the solutions obtained by running SGD using small and large batch sizes, respectively[3]. Using the linear interpolation approach

[3] In this section, we consider the "running mean" and "running variance" as trainable parameters and include them in θ. Note that the original study by Goodfellow et al. [13] does not consider batch normalization. These parameters are not included in θ in future sections, as they are only needed when interpolating between two minimizers.



(a) 7.37% 11.07% (b) $\|\theta\|_2$, WD=0 (c) WD=0

(d) 6.0% 10.19% (e) $\|\theta\|_2$, WD=5e-4 (f) WD=5e-4

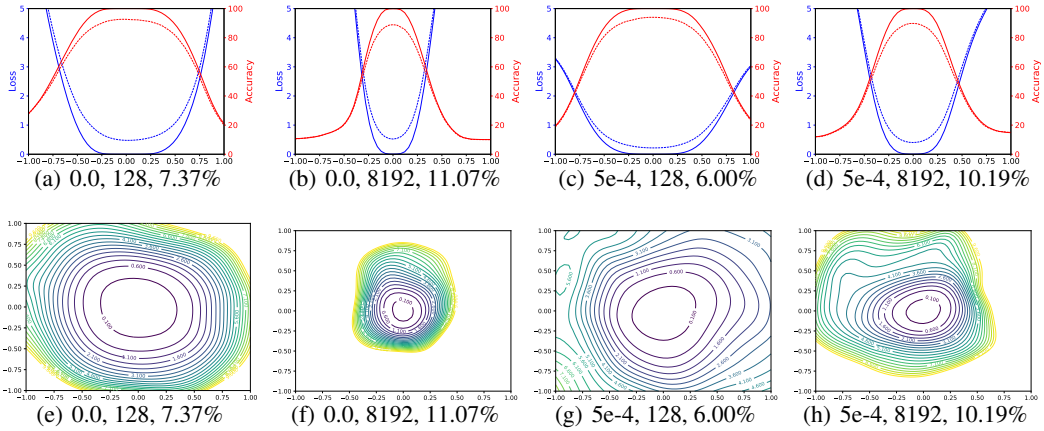
Figure 2: (a) and (d) are the 1D linear interpolation of VGG-9 solutions obtained by small-batch and large-batch training methods. The blue lines are loss values and the red lines are accuracies. The solid lines are training curves and the dashed lines are for testing. Small batch is at abscissa 0, and large batch is at abscissa 1. The corresponding test errors are shown below. (b) and (e) shows the change of weights norm $\|\theta\|_2$ during training. When weight decay is disabled, the weight norm grows steadily during training without constraints (c) and (f) are the weight histograms, which verify that small-batch methods produce more large weights with zero weight decay and more small weights with non-zero weight decay.

[13], we plot the loss values on both training and testing data sets of CIFAR-10, along a direction containing the two solutions, i.e., $f(\alpha) = L(\theta^s + \alpha(\theta^l - \theta^s))$.

Figure 2(a) shows linear interpolation plots with θ^s at x-axis location 0, and θ^l at location 1. As observed by [24], we can clearly see that the small-batch solution is quite wide, while the large-batch solution is sharp. However, this sharpness balance can be flipped simply by turning on weight decay [25]. Figure 2(d) show results of the same experiment, except this time with a non-zero weight decay parameter. This time, the large batch minimizer is considerably flatter than the sharp small batch minimizer. However, we see that small batches generalize better in all experiments; there is no apparent correlation between sharpness and generalization. We will see that these sharpness comparisons are extremely misleading, and fail to capture the endogenous properties of the minima.

The apparent differences in sharpness can be explained by examining the weights of each minimizer. Histograms of the network weights are shown for each experiment in Figure 2(c) and (f). We see that, when a large batch is used with zero weight decay, the resulting weights tend to be smaller than in the small batch case. We reverse this effect by adding weight decay; in this case the large batch minimizer has much larger weights than the small batch minimizer. This difference in scale occurs for a simple reason: A smaller batch size results in more weight updates per epoch than a large batch size, and so the shrinking effect of weight decay (which imposes a penalty on the norm of the weights) is more pronounced. The evolution of the weight norms during training is depicted in Figure 2(b) and (e). Figure 2 is not visualizing the endogenous sharpness of minimizers, but rather just the (irrelevant) weight scaling. The scaling of weights in these networks is irrelevant because batch normalization re-scales the outputs to have unit variance. However, small weights still appear more sensitive to perturbations, and produce sharper looking minimizers.

Filter Normalized Plots We repeat the experiment in Figure 2, but this time we plot the loss function near each minimizer separately using random filter-normalized directions. This removes the apparent differences in geometry caused by the scaling depicted in Figure 2(c) and (f). The results, presented in Figure 3, still show differences in sharpness between small batch and large batch minima, however these differences are much more subtle than it would appear in the un-normalized plots. For comparison, sample un-normalized plots and layer-normalized plots are shown in Section A.2 of



(a) 0.0, 128, 7.37% (b) 0.0, 8192, 11.07% (c) 5e-4, 128, 6.00% (d) 5e-4, 8192, 10.19%

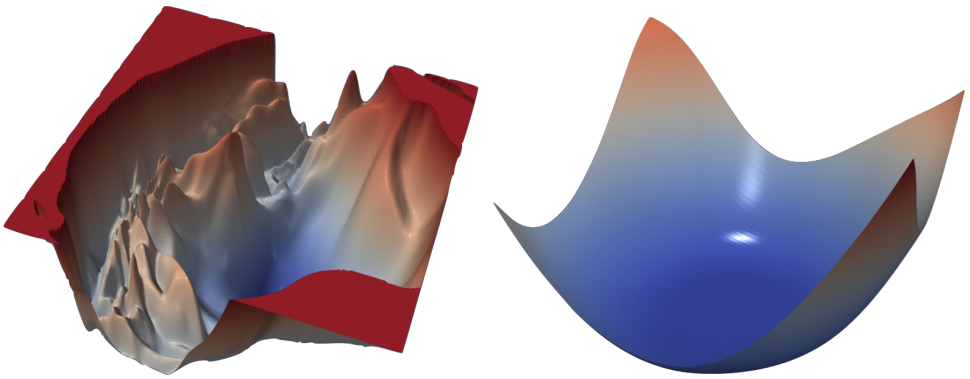
(e) 0.0, 128, 7.37% (f) 0.0, 8192, 11.07% (g) 5e-4, 128, 6.00% (h) 5e-4, 8192, 10.19%

Figure 3: The 1D and 2D visualization of solutions obtained using SGD with different weight decay and batch size. The title of each subfigure contains the weight decay, batch size, and test error.

the Appendix. We also visualize these results using two random directions and contour plots. The weights obtained with small batch size and non-zero weight decay have wider contours than the sharper large batch minimizers. Results for ResNet-56 appear in Figure 15 of the Appendix. Using the filter-normalized plots in Figure 3, we can make side-by-side comparisons between minimizers, and we see that now sharpness correlates well with generalization error. Large batches produced visually sharper minima (although not dramatically so) with higher test error.

6 What Makes Neural Networks Trainable? Insights on the (Non)Convexity Structure of Loss Surfaces

Our ability to find global minimizers to neural loss functions is not universal; it seems that some neural architectures are easier to minimize than others. For example, using skip connections, the authors of [17] trained extremely deep architectures, while comparable architectures without skip connections are not trainable. Furthermore, our ability to train seems to depend strongly on the initial parameters from which training starts. Using visualization methods, we do an empirical study of neural architectures to explore why the non-convexity of loss functions seems to be problematic in some situations, but not in others. We aim to provide insight into the following questions: Do loss functions have significant non-convexity at all? If prominent non-convexities exist, why are they not problematic in all situations? Why are some architectures easy to train, and why are results so sensitive to the initialization? We will see that different architectures have extreme differences in non-convexity structure that answer these questions, and that these differences correlate with generalization error.



(a) ResNet-110, no skip connections (b) DenseNet, 121 layers

Figure 4: The loss surfaces of ResNet-110-noshort and DenseNet for CIFAR-10.